

AXIS FINANCE

Backend & Database Requirements

Version 1.0 · May 2025
Architecture: FastAPI Monolith · PostgreSQL · Docker

Layer	Technology	Notes
API Framework	FastAPI (Python 3.10+)	Async, modular routers per feature
Database	PostgreSQL 15 via Docker	Primary data store
ORM	SQLAlchemy 2.0	Async sessions, Alembic migrations
Auth	Google OAuth 2.0	Single user now, multi-user ready
Market Data	Alpha Vantage (free tier)	Live investment prices
Import	CSV Parser (pandas)	Bank statement ingestion
Containerization	Docker + Docker Compose	All services in one compose file

Axis Finance uses a modular monolith pattern — a single FastAPI application organized into strict feature modules. Each module owns its own router, models, schemas, and service layer. This allows fast development now while making it straightforward to extract individual modules into separate microservices when moving to multi-user at scale.

1.1 Folder Structure

The backend is organized as follows:

Path	Purpose
app/	Application root
app/main.py	FastAPI app factory, router registration, CORS, startup events
app/core/	Shared config, database session, auth middleware, dependencies
app/core/config.py	Environment variables (DB URL, API keys, OAuth secrets)
app/core/database.py	SQLAlchemy async engine + session factory
app/core/auth.py	Google OAuth flow, session validation
app/modules/	One folder per feature module
app/modules/transactions/	Transactions router, models, schemas, service, csv_parser
app/modules/budget/	Budget planner router, models, schemas, service
app/modules/investments/	Investments router, models, schemas, service, price_fetcher
app/modules/analysis/	Analysis router, service (reads from other modules)
app/modules/settings/	User settings router, models, schemas, service
app/modules/auth/	Auth router, Google OAuth service
alembic/	Database migrations
docker-compose.yml	PostgreSQL + FastAPI services
.env	Environment variables (never commit to git)

1.2 Module Rules (Enforce These Always)

- Each module MUST only import from app/core/ or its own folder
- Cross-module data access MUST go through service layer calls, never direct model imports
- Each module has its own Alembic migration file
- No business logic in routers — routers call services, services call models
- Every module is independently testable in isolation

PostgreSQL is the primary data store. All tables include created_at and updated_at timestamps. The schema is designed with a user_id foreign key on every table so multi-user support can be enabled by simply enforcing that column in queries.

2.1 users

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	Generated by DB
google_id	VARCHAR(255)	UNIQUE NOT NULL	From Google OAuth
email	VARCHAR(255)	UNIQUE NOT NULL	
name	VARCHAR(255)	NOT NULL	Display name
avatar_url	TEXT	NULLABLE	Google profile photo
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Auto-updated

2.2 transactions

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
user_id	UUID	FK → users.id	Multi-user ready
date	DATE	NOT NULL	Real transaction date
effective_date	DATE	NOT NULL	Rollover-adjusted date
category	VARCHAR(100)	NOT NULL	Food, Income, Shopping etc.
description	TEXT	NOT NULL	Merchant or note
amount	NUMERIC(12,2)	NOT NULL	Positive = credit, negative = debit
type	VARCHAR(20)	NOT NULL	credit debit
is_shifted	BOOLEAN	DEFAULT FALSE	True if effective_date ≠ date
shift_override	BOOLEAN	DEFAULT FALSE	True if user manually overrode shift
import_source	VARCHAR(50)	NULLABLE	csv manual plaid (future)
import_batch_id	UUID	NULLABLE	Groups CSV import batches
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMP	NOT NULL	

Column	Type	Constraints	Notes
		DEFAULT NOW()	

2.3 budget_categories

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
user_id	UUID	FK → users.id	
name	VARCHAR(100)	NOT NULL	Housing, Food, Salary etc.
type	VARCHAR(20)	NOT NULL	income expense savings
icon	VARCHAR(10)	NULLABLE	Emoji icon
is_fixed	BOOLEAN	DEFAULT FALSE	Fixed costs like rent
sort_order	INTEGER	DEFAULT 0	Display order in grid
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	

2.4 budget_values

Stores planned (budget) and actual amounts per category per month per year.

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
user_id	UUID	FK → users.id	
category_id	UUID	FK → budget_categories.id	
year	INTEGER	NOT NULL	e.g. 2024
month	INTEGER	NOT NULL	1–12
planned_amount	NUMERIC(12,2)	DEFAULT 0	What user planned
actual_amount	NUMERIC(12,2)	DEFAULT 0	Computed from transactions
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	

2.5 investments

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	

Column	Type	Constraints	Notes
user_id	UUID	FK → users.id	
symbol	VARCHAR(20)	NOT NULL	AAPL, BTC, VDY.TO etc.
name	TEXT	NOT NULL	Full company/asset name
shares	NUMERIC(18,8)	NOT NULL	Supports crypto decimals
asset_type	VARCHAR(20)	NOT NULL	stock etf crypto bond cash
liquidity	VARCHAR(20)	NOT NULL	liquid semi-liquid locked
purchase_price	NUMERIC(12,2)	NOT NULL	Average cost basis
current_price	NUMERIC(12,2)	NULLABLE	Updated by price fetcher
price_updated_at	TIMESTAMP	NULLABLE	Last Alpha Vantage fetch
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMP	NOT NULL DEFAULT NOW()	

2.6 user_settings

Column	Type	Constraints	Notes
id	UUID	PRIMARY KEY	
user_id	UUID	FK → users.id UNIQUE	One row per user
currency	VARCHAR(10)	DEFAULT 'CAD'	
date_format	VARCHAR(20)	DEFAULT 'MM/DD/YYYY'	
fiscal_year_start	INTEGER	DEFAULT 1	Month 1–12
shift_late_income	BOOLEAN	DEFAULT TRUE	Income rollover toggle
income_cutoff_day	INTEGER	DEFAULT 20	Day of month for rollover
fi_target_override	NUMERIC(14,2)	NULLABLE	Manual FI target, else 4% rule
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMP	NOT NULL DEFAULT NOW()	

All endpoints are prefixed with `/api/v1/`. All endpoints except `/auth/*` require a valid session. Responses follow a consistent envelope: `{ data, error, meta }`.

3.1 Auth Module — `/api/v1/auth`

Method	Endpoint	Description
GET	<code>/auth/google</code>	Redirect to Google OAuth consent screen
GET	<code>/auth/google/callback</code>	Handle OAuth callback, create/update user, set session cookie
POST	<code>/auth/logout</code>	Clear session cookie
GET	<code>/auth/me</code>	Return current authenticated user profile

3.2 Transactions Module — `/api/v1/transactions`

Method	Endpoint	Description
GET	<code>/transactions</code>	List all transactions. Supports filters: category, type, date_from, date_to, search, page, page_size
POST	<code>/transactions</code>	Create single transaction. Backend auto-calculates effective_date based on user settings
GET	<code>/transactions/:id</code>	Get single transaction by ID
PUT	<code>/transactions/:id</code>	Update transaction. If shift_override=true, skip rollover logic for this transaction
DELETE	<code>/transactions/:id</code>	Soft delete transaction (set deleted_at)
POST	<code>/transactions/import/csv</code>	Upload CSV file. Returns import preview with parsed + auto-categorized rows
POST	<code>/transactions/import/confirm</code>	Confirm and persist a CSV import batch by import_batch_id
GET	<code>/transactions/summary</code>	Return total income, total expenses, net flow for a given month/year

3.3 Budget Module — `/api/v1/budget`

Method	Endpoint	Description
GET	<code>/budget/categories</code>	List all budget categories (income, expense, savings)
POST	<code>/budget/categories</code>	Create new budget category
PUT	<code>/budget/categories/:id</code>	Update category name, icon, is_fixed, sort_order
DELETE	<code>/budget/categories/:id</code>	Delete category (only if no associated values)

Method	Endpoint	Description
GET	/budget/values	Get full budget grid for a year. Returns planned + actual per category per month
PUT	/budget/values	Bulk upsert planned amounts. Body: array of { category_id, year, month, planned_amount }
GET	/budget/performance	Return Performance Summary: planned vs actual totals for income, expenses, savings for a year

3.4 Investments Module — /api/v1/investments

Method	Endpoint	Description
GET	/investments	List all holdings with current price, value, 24h change, allocation %
POST	/investments	Add new holding
PUT	/investments/:id	Update holding (shares, purchase_price, liquidity, asset_type)
DELETE	/investments/:id	Remove holding
GET	/investments/summary	Return portfolio value, total gain, return rate, liquid vs illiquid split
POST	/investments/refresh-prices	Trigger Alpha Vantage price fetch for all holdings. Rate-limited to once per 15 minutes
GET	/investments/performance	Return historical portfolio value data points for the performance chart

3.5 Analysis Module — /api/v1/analysis

Method	Endpoint	Description
GET	/analysis/monthly-snapshot	Return total spent, biggest category, most improved category for a given month
GET	/analysis/category-breakdown	Return per-category actual vs budget with % used for a given month
GET	/analysis/trends	Return 6-month spend history per category with trend tag (Rising/Reduced/Stable)
GET	/analysis/spending-insights	Return computed insight chips: anomalies, over-budget flags, MoM changes

3.6 Settings Module — /api/v1/settings

Method	Endpoint	Description
GET	/settings	Return current user settings
PUT	/settings	Update any settings field. income_cutoff_day change triggers recalculation of all effective_dates

4.1 Income Rollover Rule

This is the most important piece of business logic in the entire backend. It must be implemented correctly and consistently.

- When a transaction is of type 'credit' (income) AND its date is $\geq$ income_cutoff_day (default: 20th) of the month
 - The backend automatically sets effective_date = 1st day of the following month
 - The is_shifted flag is set to TRUE on that transaction
 - If shift_override = TRUE on the transaction, skip this logic and use date as effective_date
 - All dashboard metrics, budget planner actuals, and analysis use effective_date — never date
 - The Ledger/Transactions table shows the real date with a 'Shifted' badge when is_shifted = TRUE
-
- On every new transaction POST — calculate effective_date before insert
 - On transaction PUT — recalculate unless shift_override is explicitly set
 - When user changes income_cutoff_day in settings — recalculate ALL historical transactions
 - When user toggles shift_late_income OFF in settings — set effective_date = date for all, is_shifted = FALSE for all

4.2 Budget Actual Computation

The actual_amount in budget_values is never manually set — it is always computed from transactions.

- For expense categories: sum of ABS(amount) for transactions where effective_date falls in that month/year and category matches
- For income categories: sum of amount for credit transactions where effective_date falls in that month/year and category matches
- For savings categories: treat as expense (money going out to savings accounts)
- This computation runs on-demand when GET /budget/values is called — not stored statically
- Alternatively, update actual_amount as a background task whenever a transaction is created/updated/deleted

4.3 CSV Import Pipeline

- Accept multipart/form-data CSV file upload
- Detect delimiter (comma or semicolon)
- Read headers, return first 5 rows as preview with detected column names

- Return a session token (import_batch_id) tied to the parsed data in memory or temp storage
- User sends column mapping: { date_col, description_col, amount_col, type_col? }
- Backend parses all rows using the mapping
- Auto-categorize each row by matching description against keyword rules:
 - *Walmart, Whole Foods, Loblaws, Metro, Sobeys* → *Food*
 - *Netflix, Spotify, Disney+, Apple TV* → *Subscriptions*
 - *Shell, Esso, Petro-Canada, Gas, Uber, Lyft* → *Transportation*
 - *Hydro, Electric, Internet, Rogers, Bell, Telus* → *Utilities*
 - *Salary, Payroll, Direct Deposit, Payroll* → *Income*
 - *Amazon, Apple.com, Best Buy* → *Shopping*
 - *Pharmacy, Shoppers, Rexall* → *Health*
 - *Unmatched* → *Uncategorized*
- Run duplicate detection: flag rows where date + amount + description match an existing transaction
- Return full parsed preview with categories, amounts, types, and duplicate flags
- User confirms, optionally sending category overrides per row
- Backend inserts all non-duplicate rows (or all if user chose to include duplicates)
- Apply rollover logic to each imported transaction
- Return { imported: N, skipped: N, import_batch_id }

4.4 Live Investment Prices (Alpha Vantage)

- Price fetcher is a standalone service module: app/modules/investments/price_fetcher.py
- On POST /investments/refresh-prices, fetch current price for each holding symbol from Alpha Vantage
- Alpha Vantage free tier: 25 API calls/day, 5 calls/minute — implement rate limiting and request queuing
- For stocks/ETFs: use GLOBAL_QUOTE endpoint
- For crypto (BTC, ETH etc.): use CURRENCY_EXCHANGE_RATE endpoint
- Store current_price and price_updated_at on each investment row after fetch
- If fetch fails for a symbol, keep last known price, log the error, do not crash other symbols
- Auto-refresh trigger: on app startup if price_updated_at is older than 24 hours

4.5 Dashboard Metric Computations

- $\text{runway_months} = \text{total_liquid_cash} / \text{avg_monthly_spend}$
- $\text{total_liquid_cash} = \text{sum of liquid investment values} + (\text{total income} - \text{total expenses}) \text{ for last 3 months}$
- $\text{avg_monthly_spend} = \text{average of last 3 months of total expense (using effective_date)}$

Auth is implemented as its own module (app/modules/auth/) so it can be extracted into a separate service when going multi-user. The current implementation uses Google OAuth 2.0.

5.1 Google OAuth Flow

- GET /auth/google — generate OAuth URL with state parameter, redirect user to Google
- GET /auth/google/callback — receive code from Google, exchange for tokens, get user profile
- If user exists (match by google_id): update name/avatar, issue session
- If user does not exist: create user row, create default user_settings row, issue session
- Session stored as an HTTP-only secure cookie (not localStorage)
- Session token is a signed JWT containing user_id and expiry (7 days)

5.2 Multi-User Readiness

- Every DB query in every module MUST filter by user_id from the session
- user_id is never trusted from the request body — always taken from the validated session
- This means adding a second user later requires zero schema changes
- JWT auth can be added alongside Google OAuth later without replacing it

5.3 Required Environment Variables

Variable	Description
GOOGLE_CLIENT_ID	From Google Cloud Console OAuth credentials
GOOGLE_CLIENT_SECRET	From Google Cloud Console OAuth credentials
GOOGLE_REDIRECT_URI	Must match exactly: http://localhost:3000/api/v1/auth/google/callback
SESSION_SECRET_KEY	Random 32-byte string for signing session JWTs
DATABASE_URL	postgresql+asyncpg://user:pass@localhost:5432/axisfinance
ALPHA_VANTAGE_API_KEY	From alphavantage.co free account
CORS_ORIGINS	http://localhost:5173 (Vite dev server)

The entire backend runs via Docker Compose. Two services: the FastAPI app and PostgreSQL. The frontend Vite dev server runs separately on the host.

6.1 docker-compose.yml Services

Service	Image	Port	Purpose
db	postgres:15-alpine	5432	Primary PostgreSQL database
api	Custom Dockerfile	3000	FastAPI application server

6.2 Key Docker Requirements

- PostgreSQL data persisted via a named Docker volume (axisfinance_pgdata)
- API container depends_on db with health check before starting
- Alembic migrations run automatically on API container startup
- .env file mounted into API container — never baked into the image
- Hot reload enabled in development via volume mount of app/ folder
- CORS configured to allow http://localhost:5173 (Vite frontend)

7.1 Modularity Rules

- Each module folder is self-contained: router.py, models.py, schemas.py, service.py
- No circular imports between modules
- Shared utilities go in app/core/ only
- Each module registers its own APIRouter, mounted in main.py
- Adding a new feature = adding a new folder in app/modules/ — nothing else changes

7.2 Error Handling

- All endpoints return consistent JSON: { data: null, error: { code, message } } on failure
- HTTP 400 for validation errors (Pydantic), 401 for unauthenticated, 403 for unauthorized, 404 for not found, 500 for server errors
- Never expose raw database errors or stack traces to the client
- Log all 500 errors with full traceback to stdout (Docker captures this)

7.3 Data Validation

- All request bodies validated with Pydantic v2 schemas
- Amount fields: must be non-zero numeric, max 12 digits, 2 decimal places
- Date fields: must be valid dates, not in the future for transactions
- Category names: max 100 characters, no special characters
- Symbol fields: uppercase only, max 20 characters

7.4 Performance

- All DB queries use async SQLAlchemy sessions (never blocking)
- Budget actual computation: cache result for 60 seconds to avoid recomputing on every request
- Alpha Vantage calls: never block a user-facing request — run as background task
- CSV imports: process in chunks of 500 rows for large files

7.5 Testing

- Each module has a tests/ subfolder
- Minimum test coverage: all service layer functions
- Use pytest + pytest-asyncio for async test support
- Use a separate test database (SQLite in-memory or test PostgreSQL schema)
- CI: run tests on every git push before any deployment